Constant Pointer View - `std::as_const` Strikes Back!

ISO/IEC JTC1 SC22 WG21 P1011R0


```
ADAM David Alan Martin  (adam@recursive.engineer)
                        (adam.martin@mongodb.com)
```


Table of Contents

Introduction

In "Constant View: A proposal for a `std::as_const` helper function template" (N4380), I (and Alisdair Meredith) proposed a utility for the standard library to afford constant-view of a specific value, by reference. Since its adoption into C++17, we've learned a lot about how this utility can be used, and we've discovered a new set of cases where a similar utility might be useful. This paper outlines that similar utility and the motivations for it.

We propose a new helper function template, `std::as_ptr_const`, which would live in the `<utility>` header. A simple example usage:

```
#include <utility>
#include <type_traits>

void
demoUsage()
{
    std::string mutableString= "Hello World!";
    std::string *mutableStringPtr= &mutableString
    const std::string &constPtrView= std::as_ptr_const( mutableStringPtr );

    assert( &constPtrView == mutableStringPtr );
    assert( std::as_ptr_const( mutableStringPtr ) == &mutableString );

    assert( &*constPtrView == &std::as_const( mutableString ) );
    assert( *constPtrView == std::as_const( mutableString ) );
}
```

Background

The C++ Language distinguishes between `const Type *` and `Type *` in ADL lookup for selecting function overloads. The selection of overloads can occur among functions like:

```
int processEmployees( std::vector< Employee > *employeeList );
bool processEmployees( const std::vector< Employee > *employeeList );
```

Oftentimes these functions should have the same behavior, but sometimes free (or member) functions will return different types, depending upon which qualifier (const or non-const) applies to the source type. Further, the semantics of such functions could differ drastically, depending upon whether the function knows that it has permission to modify the memory that was referenced.

Motivation

This proposal shares essentially the same motivation as N4380, just expanded to include pointers, beyond references. It would be a breaking change to add an overload to `std::as_const` which handled pointers differently. There are legitimate use cases for the `std::as_const< T *& >` case which would collide with an overload to effect this semantic. Therefore I propose a distinct name for this operation.

Alternatives

1. Consider developing a more generic "safe-const-cast" language idiom
2. Develop a more generic library utility for `T **`, `T ***`, and member pointers

Discussion

This conversion, or alternative-viewpoint, is always safe. Following the rule that safe and common actions shouldn't be ugly while dangerous and infrequent actions should be hard to write, we can conclude that this addition of const is an operation that should not be ugly and hard. Const-cast syntax is ugly and therefore its usage is inherently eschewed.

We have deliberately chosen not to overload `std::as_const` for pointer cases, as there are legitimate use cases for references to pointer which may exist today that shouldn't be broken. Further, since the target of the pointer is the type being augmented with const, not the pointer itself, an overload which declines to modify rvalues is not necessary, as pointers have different semantics than references, when it comes to copying and lifetime extension rules.

Proposed Implementation

In the `<utility>` header, the following code should work:

```
// ...
// -- Assuming that the file has reverted to the global namespace --
namespace std
{
    template< typename T >
    std::add_const< T >_t *
    as_ptr_const( T *const t ) noexcept
    {
        return t;
    }
}
// ...
```

Further Discussion

The return of my request to have a constant-view mechanism in the language or the library takes the form of an operation over an expanded set of possible types. It is probably worth investigating a more general feature for this purpose, given that we've identified another set of types which would benefit from a constant-view mechanism. I request guidance from LEWG and LWG on whether there is value in further investigatation of this kind of idea, and whether it should take the form of a language feature or a library feature.

Acknowledgements

Thanks to Alisdair Meredith for his help with the original `std::as_const` facility and for his input and feedback on this paper.